

"Exploring ICMP Tunneling with ptunnel: A Practical Demonstration"

Name:-Pruthak Kansara

Date:-27/11/2024

Instructor Name:-Krapesh Bhatt

Table of Contents

1.Introduction

2.Objective

3.Tools and Technologies

4.Ptunnel installation and usage

5.Execution

6.Traffic Analysis

7.Conclusion

Introduction

ICMP tunneling is a technique used to bypass network restrictions by encapsulating data within ICMP packets. This project demonstrates the use of `ptunnel` to establish an ICMP tunnel between two virtual machines, enabling data transmission even when other communication channels are restricted. The exercise highlights the importance of understanding such techniques to enhance network security and detect potential vulnerabilities.

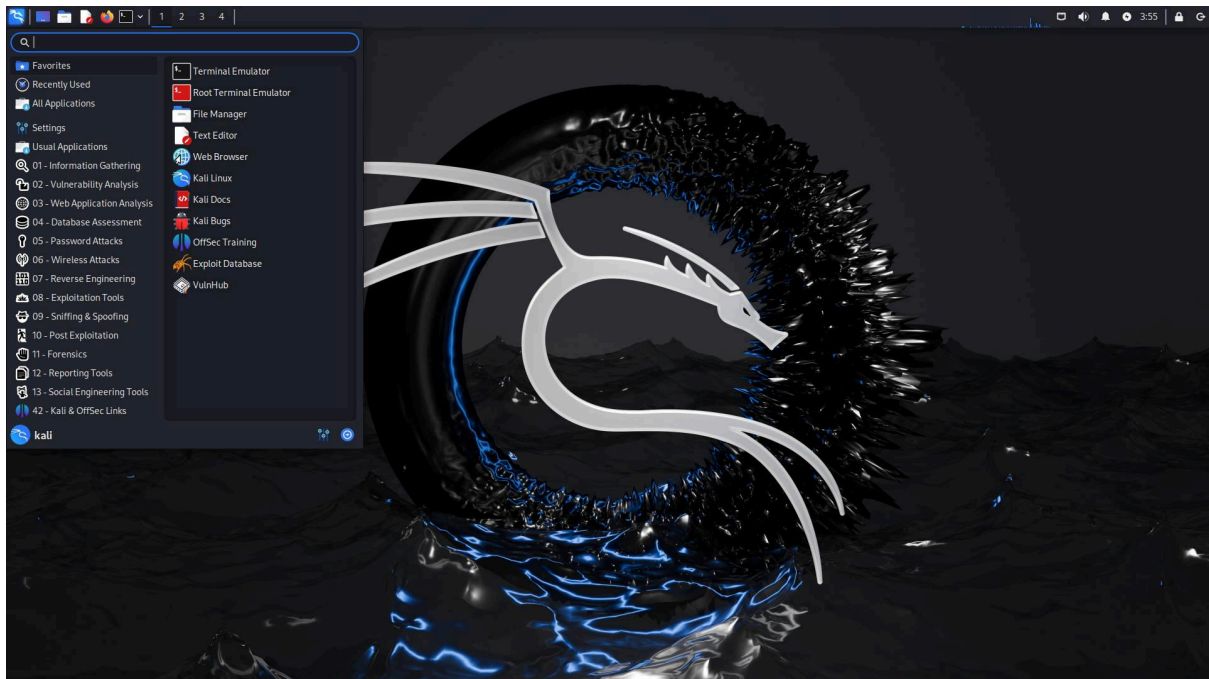
Objective

The main goal of this project is to demonstrate ICMP tunneling in a controlled network environment to simulate data exfiltration and analyze its implications on network security. By the end of this project, the following objectives will be achieved:

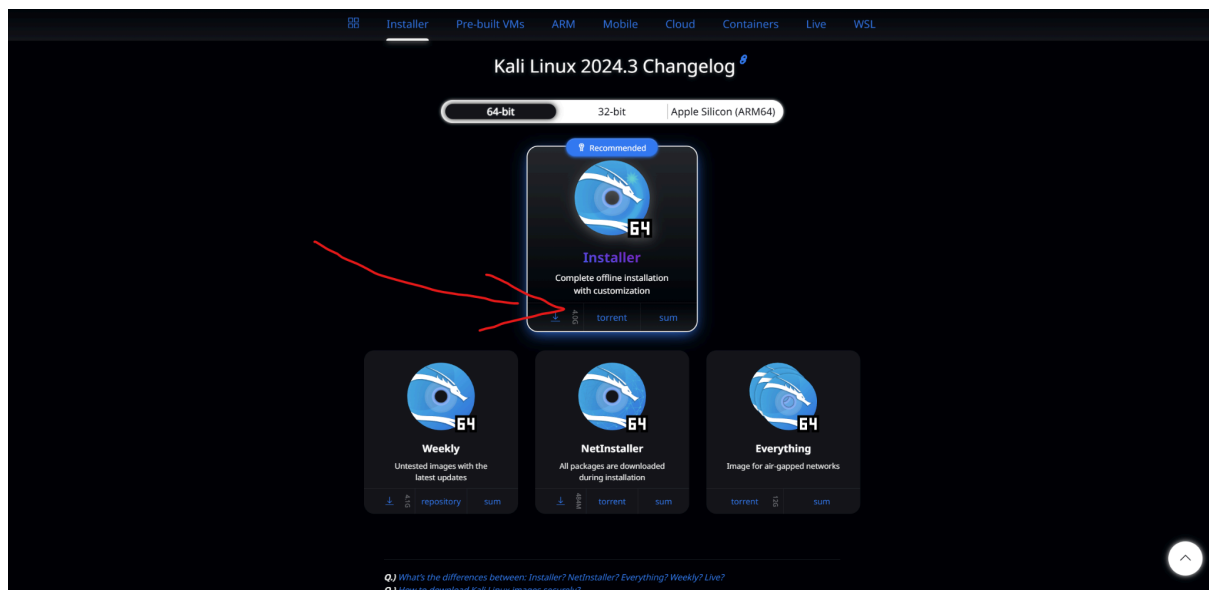
1. Set up Virtual Machines for Kali Linux (attacker) and Ubuntu Server (victim).
2. Configure firewall rules on the victim machine to allow only ICMP and SSH traffic.
3. Install and configure `ptunnel` on both the attacker and victim machines.
4. Establish an ICMP tunnel to forward traffic from the attacker to the victim machine.
5. Simulate data exfiltration over the ICMP tunnel and establish an SSH session.
6. Monitor and analyze ICMP traffic using Wireshark for detailed insights.

Tools and Technologies

1.Kali linux



→We will use Kali Linux as our attacker machine and install it on VirtualBox, referring to it as VM1.



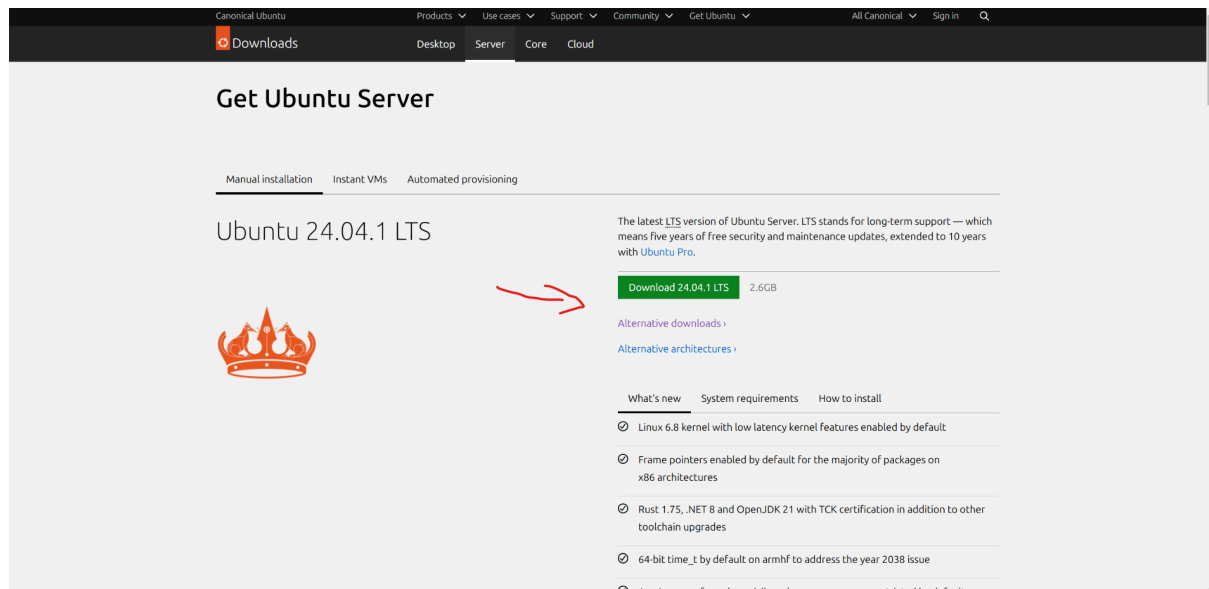
→ You can download Kali Linux using the following link: [Kali Linux Download](#)

→ Make sure to select the correct Kali ISO file that matches your system requirements.

1.Ubuntu

```
chroot: can't execute 'glib-compile-schemas': No such file or directory
Using CD-ROM mount point /cdrom/
Identifying... [92043204992947830abda80987b766a7-2]
Scanning disc for index files...
Found 2 package indexes, 0 source indexes, 0 translation indexes and 1 signatures
Found label 'Ubuntu-Server 18.04 LTS _Bionic Beaver_ - Beta amd64 (20180404)'
This disc is called:
'Ubuntu-Server 18.04 LTS _Bionic Beaver_ - Beta amd64 (20180404)'
Copying package lists...[ TIME ] Timed out waiting for device dev-disk-by\x2duuid-00c629d6\x2d06ab\x2d4dfd\x2db21e\x2dc3186f34105d.device.
[DEPEND] Dependency failed for /subiquity_config.
[ OK ] Started Uncomplicated firewall.
[ OK ] Started Create list of required static device nodes for the current kernel.
        Starting Create Static Device Nodes in /dev...
[ OK ] Mounted POSIX Message Queue File System.
[ OK ] Mounted Kernel Debug File System.
[ OK ] Started Remount Root and Kernel File Systems.
        Starting Load/Save Random Seed...
[ OK ] Mounted Huge Pages File System.
[ OK ] Started Load/Save Random Seed.
[ OK ] Started Create Static Device Nodes in /dev.
        Starting udev Kernel Device Manager...
[ OK ] Started Journal Service.
        Starting Flush Journal to Persistent Storage...
[ OK ] Started udev Kernel Device Manager.
[ OK ] Started Load Kernel Modules.
        Mounting Kernel Configuration File System...
        Mounting FUSE Control File System...
        Starting Apply Kernel Variables...
[ OK ] Started Flush Journal to Persistent Storage.
[ OK ] Mounted FUSE Control File System.
[ OK ] Mounted Kernel Configuration File System.
[ OK ] Started LVM2 metadata daemon.
[ OK ] Started udev Coldplug all Devices.
[ OK ] Started Monitoring of LVM2 mirrors, snapshots etc. using dmeventd or progress polling.
[ OK ] Started Apply Kernel Variables.
```

→We will use Ubuntu Server as our victim machine, referring to it as VM2.



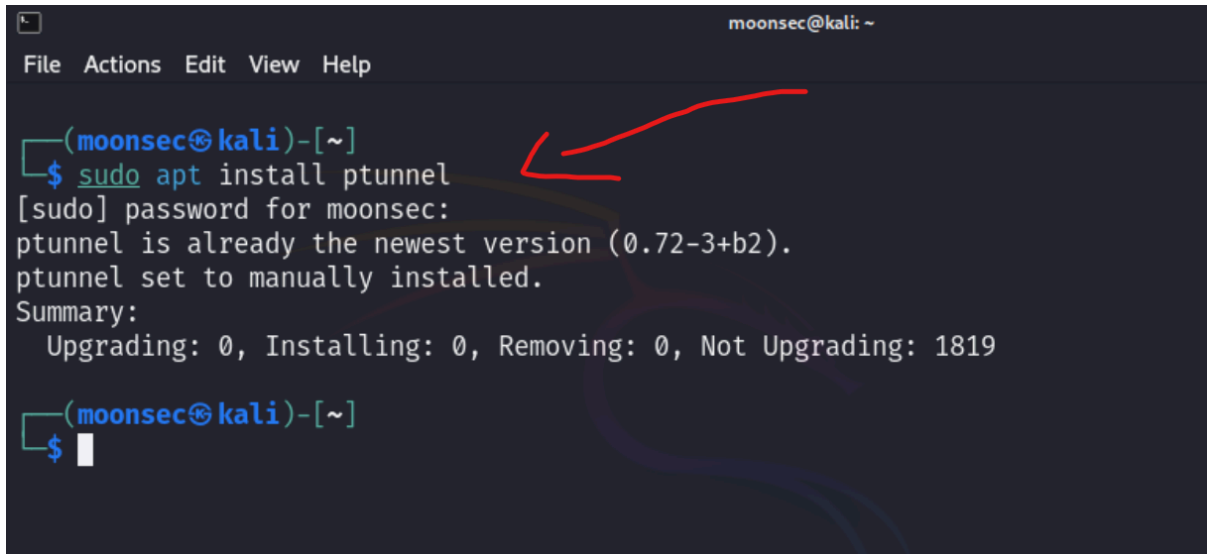
→ You can download Ubuntu Server from this link:
[Ubuntu Server Download](#).

→Now, for the tool, we will use Ptunnel. You can learn more about Ptunnel through this link:[Ptunnel Documentation](#), as we will demonstrate ICMP tunneling using Ptunnel.

→For analysis and capturing packets to better understand how things work in the background, we will use Wireshark to capture the packets.

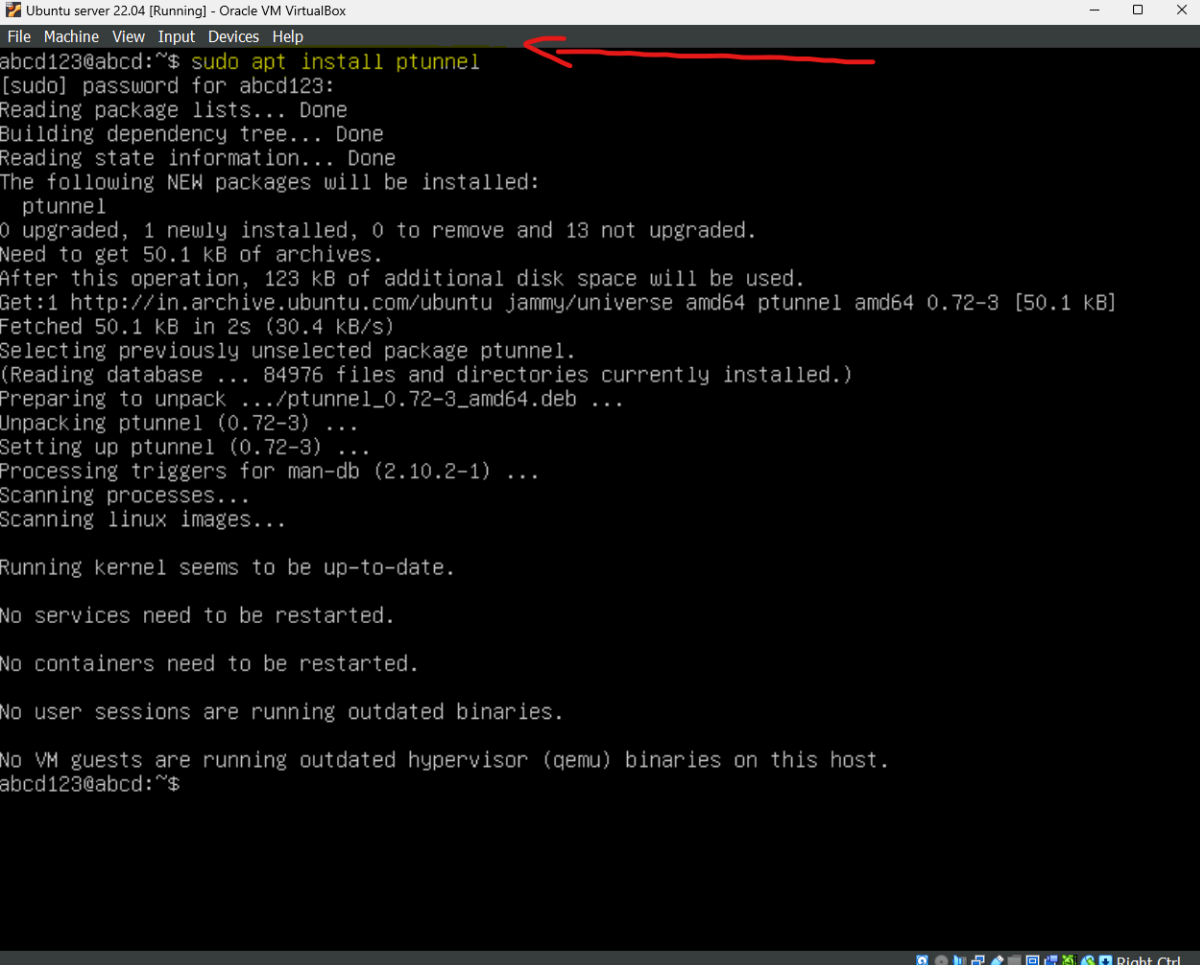
Ptunnel installation and usage

→ Now, we need to install Ptunnel on both the attacker and victim machines.



A terminal window titled 'moonsec@kali: ~' with a menu bar (File, Actions, Edit, View, Help). The prompt is '(moonsec@kali)-[~]'. The command '\$ sudo apt install ptunnel' is entered. The output shows the password prompt, a confirmation that ptunnel is the newest version (0.72-3+b2), and a summary of the installation. A red arrow points to the command line. The terminal shows the following text:

```
(moonsec@kali)-[~]  
$ sudo apt install ptunnel  
[sudo] password for moonsec:  
ptunnel is already the newest version (0.72-3+b2).  
ptunnel set to manually installed.  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1819  
  
(moonsec@kali)-[~]  
$
```



```
Ubuntu server 22.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
abcd123@abcd:~$ sudo apt install ptunnel
[sudo] password for abcd123:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  ptunnel
0 upgraded, 1 newly installed, 0 to remove and 13 not upgraded.
Need to get 50.1 kB of archives.
After this operation, 123 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 ptunnel amd64 0.72-3 [50.1 kB]
Fetched 50.1 kB in 2s (30.4 kB/s)
Selecting previously unselected package ptunnel.
(Reading database ... 84976 files and directories currently installed.)
Preparing to unpack .../ptunnel_0.72-3_amd64.deb ...
Unpacking ptunnel (0.72-3) ...
Setting up ptunnel (0.72-3) ...
Processing triggers for man-db (2.10.2-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

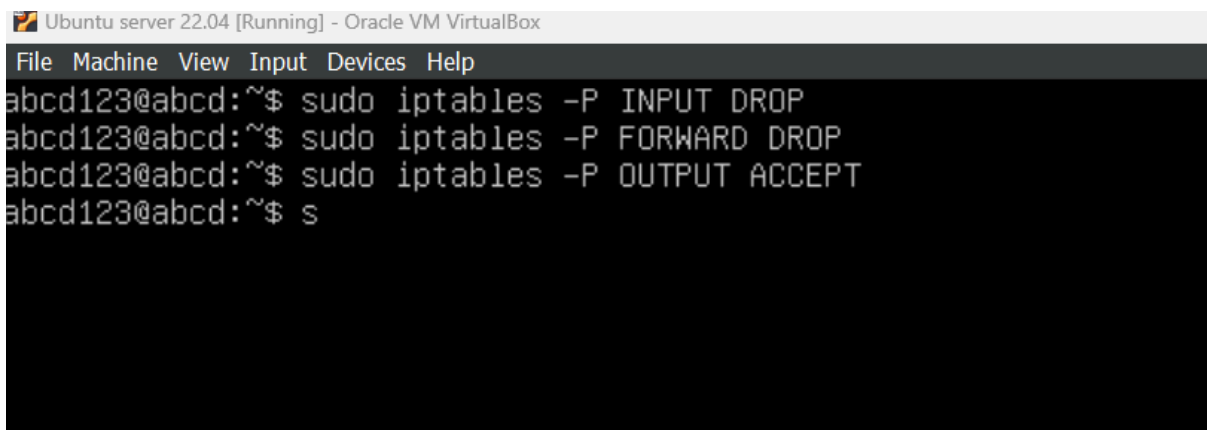
No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
abcd123@abcd:~$
```

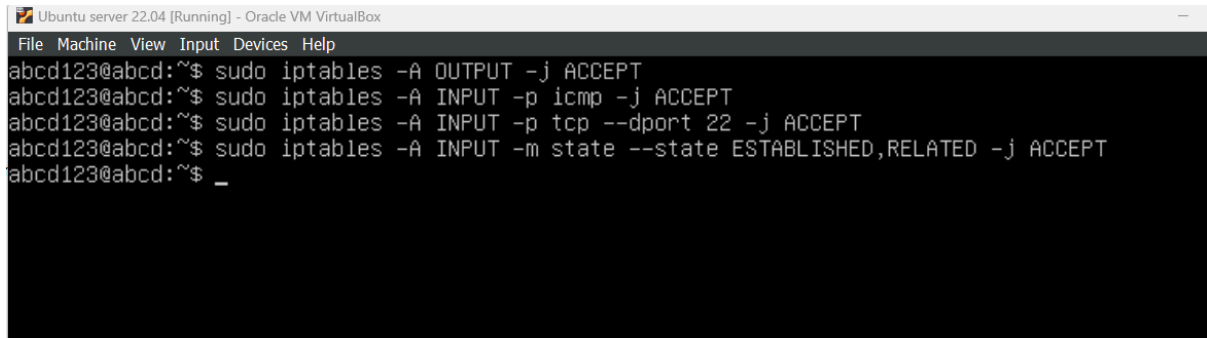
→ You can see in the above pictures that we installed Ptunnel on both VM1 and VM2 machines using the command:-**sudo apt install ptunnel**

→Now, we need to fulfill some requirements to use Ptnnel successfully on the victim machine by allowing only ICMP traffic and SSH connections, as outlined below.



```
Ubuntu server 22.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
abcd123@abcd:~$ sudo iptables -P INPUT DROP
abcd123@abcd:~$ sudo iptables -P FORWARD DROP
abcd123@abcd:~$ sudo iptables -P OUTPUT ACCEPT
abcd123@abcd:~$ s
```

Drop all incoming traffic by default
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT



```
Ubuntu server 22.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
abcd123@abcd:~$ sudo iptables -A OUTPUT -j ACCEPT
abcd123@abcd:~$ sudo iptables -A INPUT -p icmp -j ACCEPT
abcd123@abcd:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
abcd123@abcd:~$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
abcd123@abcd:~$ _
```

Allow all outgoing traffic

sudo iptables -A OUTPUT -j ACCEPT

Allow incoming ICMP (ping) traffic

sudo iptables -A INPUT -p icmp -j ACCEPT

**# Allow incoming traffic on port 22 for SSH
(needed for ptunnel)**

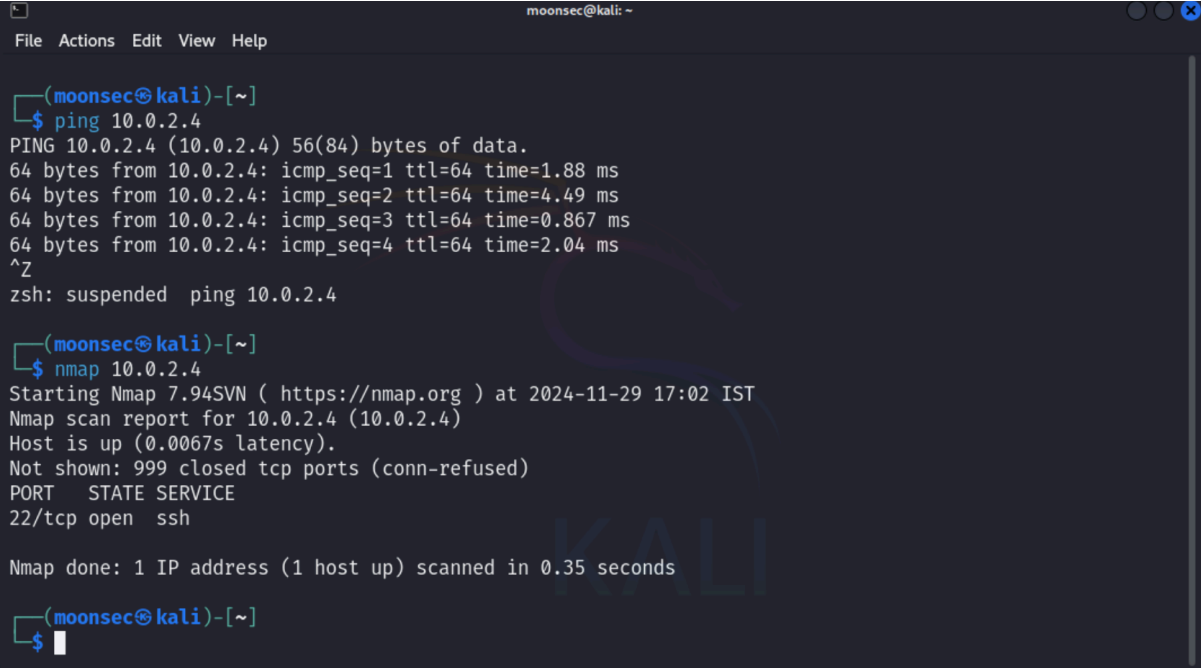
**sudo iptables -A INPUT -p tcp --dport 22 -j
ACCEPT**

Allow established connections

**sudo iptables -A INPUT -m state --state
ESTABLISHED,RELATED -j ACCEPT**

→Now, we need to verify the above step using our attacker machine.

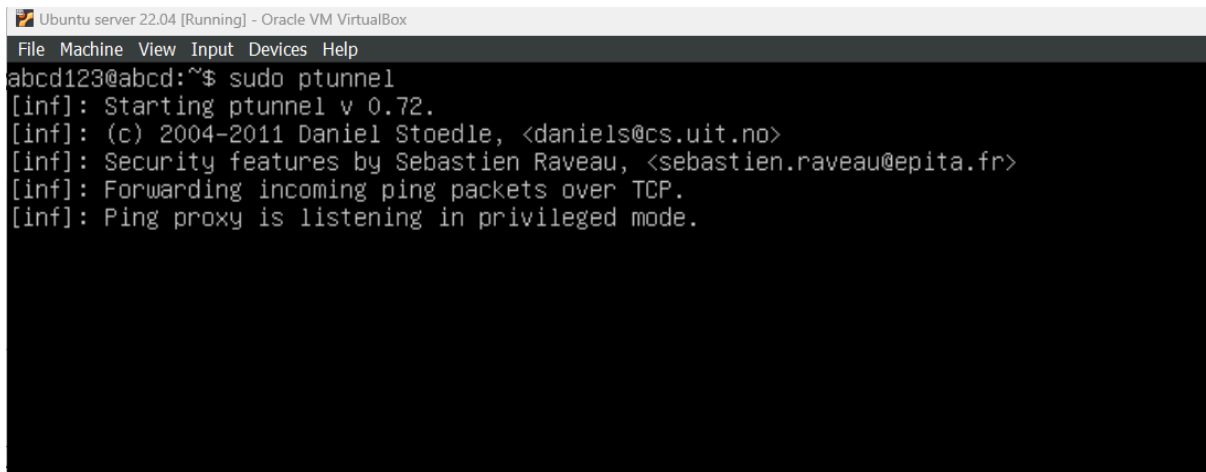
→Make sure both VMs are set to NAT network mode in VirtualBox.

A screenshot of a terminal window titled 'moonsec@kali: ~'. The window has a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the following commands and output:

```
(moonsec@kali)~[~]  
$ ping 10.0.2.4  
PING 10.0.2.4 (10.0.2.4) 56(84) bytes of data.  
64 bytes from 10.0.2.4: icmp_seq=1 ttl=64 time=1.88 ms  
64 bytes from 10.0.2.4: icmp_seq=2 ttl=64 time=4.49 ms  
64 bytes from 10.0.2.4: icmp_seq=3 ttl=64 time=0.867 ms  
64 bytes from 10.0.2.4: icmp_seq=4 ttl=64 time=2.04 ms  
^Z  
zsh: suspended ping 10.0.2.4  
  
(moonsec@kali)~[~]  
$ nmap 10.0.2.4  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-11-29 17:02 IST  
Nmap scan report for 10.0.2.4 (10.0.2.4)  
Host is up (0.0067s latency).  
Not shown: 999 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
  
Nmap done: 1 IP address (1 host up) scanned in 0.35 seconds  
  
(moonsec@kali)~[~]  
$
```

→As you can see from the above picture, I am receiving a response when performing a ping, and a scan shows that only port 22 (SSH) is open.

Execution



```
Ubuntu server 22.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
abcd123@abcd:~$ sudo ptunnel
[inf]: Starting ptunnel v 0.72.
[inf]: (c) 2004-2011 Daniel Stuedle, <daniels@cs.uit.no>
[inf]: Security features by Sebastien Raveau, <sebastien.raveau@epita.fr>
[inf]: Forwarding incoming ping packets over TCP.
[inf]: Ping proxy is listening in privileged mode.
```

→now we have to start ptunnel on vm2 on listening mode by command sudo ptunnel



```
(moonsec@kali)-[~]
$ sudo ptunnel -p 10.0.2.4 -lp 2222 -da 127.0.0.1 -dp 22
[inf]: Starting ptunnel v 0.72.
[inf]: (c) 2004-2011 Daniel Stuedle, <daniels@cs.uit.no>
[inf]: Security features by Sebastien Raveau, <sebastien.raveau@epita.fr>
[inf]: Relaying packets from incoming TCP streams.
```

→now on attacker vm we will execute command to start ptunnel the command is :-sudo ptunnel -p <target_ip> -lp 2222 -da 127.0.0.1 -dp 22

-p = target IP (VM2)

-lp = This is the local port (2222) on the attacker machine that ptunnel will listen to for incoming traffic. Once the tunnel is established, all data sent to this port will be forwarded through the ICMP tunnel.

-da 127.0.0.1 = This is the destination IP address of the service you want to connect to on the target machine (in this case, localhost for SSH)

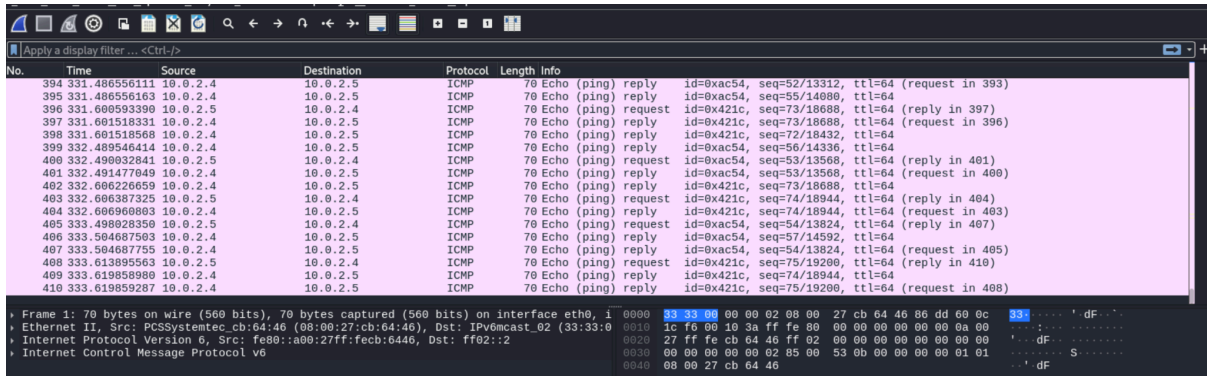
-dp 22 = This is the destination port (22) on the target machine, which is the SSH service port.

This command tunnels any traffic from port 2222 on the attacker machine to port 22 (SSH) on the target machine, using ICMP echo requests and replies

```
(moonsec@kali)~  
$ ssh -p 2222 abcd123@localhost  
abcd123@localhost's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-122-generic x86_64)
```

→Now, we will connect to the victim VM via SSH.

Traffic Analysis



The image shows a Wireshark packet capture window. The top toolbar includes icons for file operations, network analysis, and display filters. Below the toolbar, a display filter is set to 'Apply a display filter ... <Ctrl-F>'. The main packet list table shows 17 ICMP Echo (ping) packets, alternating between requests and replies. The packet details pane at the bottom shows the structure of a selected packet, including Ethernet II, Internet Protocol Version 6, and Internet Control Message Protocol (ICMP) fields.

No.	Time	Source	Destination	Protocol	Length	Info
394	331.486556111	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
395	331.486556163	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
396	331.600593390	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
397	331.601518331	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
398	331.601518568	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
399	332.489546414	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
400	332.490932841	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
401	332.491477049	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
402	332.606226659	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
403	332.606387325	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
404	332.606960883	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
405	333.490928350	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
406	333.504687583	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
407	333.504687755	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
408	333.613895563	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request
409	333.619858980	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply
410	333.619859287	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply

Frame 1: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0, i
Ethernet II, Src: PCSSystemtec_cb:64:46 (08:00:27:cb:64:46), Dst: IPv6mcast_02 (33:33:0
Internet Protocol Version 6, Src: fe80:a00:27ff:feeb:6446, Dst: ff02::2
Internet Control Message Protocol v6

→Wireshark captured the ICMP packets exchanged during the tunneling process. These packets included both ICMP Echo Requests and Echo Replies that facilitated the tunnel.

No.	Time	Source	Destination	Protocol	Length	Info
292	314.565616987	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) request id=0x421c, seq=56/14336, ttl=64 (reply in 293)
293	314.566289789	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0x421c, seq=56/14336, ttl=64 (request in 292)
294	314.51744475	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0x421c, seq=55/14088, ttl=64
295	314.532183706	PCSSystemtec_cb:64:...	PCSSystemtec_06:fb:...	ARP	60	who has 10.0.2.5? Tell 10.0.2.4
296	314.532197107	PCSSystemtec_06:fb:...	PCSSystemtec_cb:64:...	ARP	42	10.0.2.5 is at 08:00:27:06:fb:71
297	315.393915445	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) request id=0xac54, seq=36/9216, ttl=64 (reply in 299)
298	315.394513530	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0xac54, seq=36/9216, ttl=64 (request in 297)
299	315.394513673	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0x421c, seq=56/14336, ttl=64
300	315.51953169	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request id=0x421c, seq=57/14592, ttl=64 (reply in 302)
301	315.520321987	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) reply id=0x421c, seq=57/14592, ttl=64 (request in 301)
302	315.521112488	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0xac54, seq=40/10240, ttl=64
303	316.395320486	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request id=0xac54, seq=37/9472, ttl=64 (reply in 305)
304	316.395572161	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) reply id=0xac54, seq=37/9472, ttl=64 (request in 304)
305	316.396344240	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request id=0x421c, seq=58/14848, ttl=64 (reply in 307)
306	316.520344783	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) reply id=0x421c, seq=58/14848, ttl=64 (request in 306)
307	316.521274418	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) reply id=0x421c, seq=57/14592, ttl=64
308	316.521274608	10.0.2.4	10.0.2.5	ICMP	70	Echo (ping) request id=0xac54, seq=38/9728, ttl=64 (reply in 310)
309	317.397972553	10.0.2.5	10.0.2.4	ICMP	70	Echo (ping) request id=0xac54, seq=38/9728, ttl=64 (reply in 310)

Frame 301: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0
 Ethernet II, Src: PCSSystemtec_06:fb:71 (08:00:27:06:fb:71), Dst: PCSSystemtec_cb:64:40
 Internet Protocol Version 4, Src: 10.0.2.5, Dst: 10.0.2.4
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 Total Length: 56
 Identification: 0x02e9 (33513)
 0100 = Flags: 0x2, Don't fragment
 ...0 0000 0000 0000 = Fragment Offset: 0
 Time to Live: 64
 Protocol: ICMP (1)
 Header Checksum: 0x9fd3 [validation disabled]
 [Header checksum status: Unverified]
 Source Address: 10.0.2.5
 Destination Address: 10.0.2.4
 Internet Control Message Protocol

This Wireshark capture demonstrates an exchange of ICMP (Internet Control Message Protocol) packets between two devices on the network. Below are the key details observed in the analysis:

1. Packet Details

- Source Address: **10.0.2.5** (attacker machine).
- Destination Address: **10.0.2.4** (target machine).
- Protocol: ICMP (ping).
- Length: Each ICMP packet is 70 bytes.

2. Packet Inspection

- Frame 301 (Highlighted in the capture):
 - Type: ICMP Echo Request.
 - ID: **0x421c**.
 - Sequence Number: **577**.
 - TTL (Time to Live): **64**.
- The corresponding ICMP Echo Reply is seen in Frame 302, with the same ID and sequence number, confirming a successful ping response from the target.

3. Internet Protocol (IP) Layer

- Version: IPv4.
- Total Length: 56 bytes.
- Header Checksum: **0x9fd3** (validated).

4. Analysis

The capture illustrates a steady exchange of ICMP packets, simulating a typical ping operation. This data can be used to identify network reachability, latency, or potential tunneling activity if modified

Conclusion

In this project, we successfully demonstrated ICMP tunneling using ptunnel, which encapsulates data within ICMP echo requests and replies to bypass network firewalls and restrictions. The project involved simulating a network environment with Kali Linux (acting as the attacker) and Ubuntu Server (configured to receive the ICMP tunneled data). By capturing and analyzing the network traffic, we were able to observe how data could be covertly exfiltrated using ICMP packets.

Key findings from the project include:

- 1. ICMP as a Covert Channel:** ICMP tunneling enables hidden data transmission within regular network traffic, making it difficult for traditional firewalls to detect.
- 2. Network Traffic Analysis:** Using tools like Wireshark, we effectively captured and analyzed the ICMP packets, revealing the encapsulated data and understanding the underlying mechanisms of the attack.

3. Detection Challenges: Although common IDS/IPS systems may flag unusual ICMP traffic, detecting ICMP tunneling requires close inspection of packet patterns and payloads. This highlights the need for advanced network monitoring solutions.

4. Practical Application: The project reinforced the significance of securing network communication and implementing effective monitoring systems to detect malicious activities like data exfiltration via ICMP tunneling.

This project successfully showcases how ICMP tunneling can be used for covert data exfiltration, while also emphasizing the need for continuous monitoring and proper network security measures to prevent such threats. It is a useful demonstration of both offensive and defensive cybersecurity techniques.

